MVC

Suleman Shahid
suleman.shahid@lums.edu.pk

Abdul Ali Bangash
abdulali@lums.edu.pk

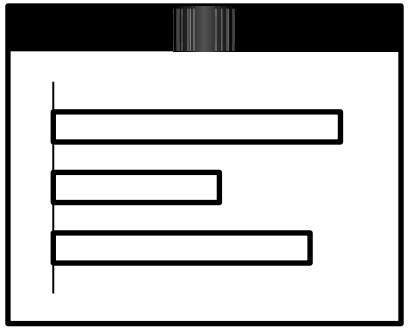
Department of Computing Science

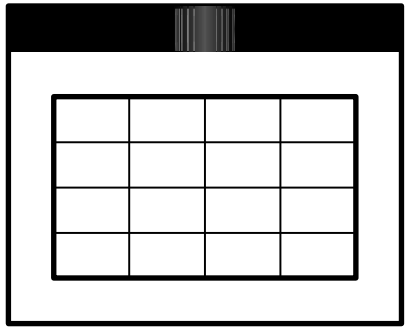
LUMS University

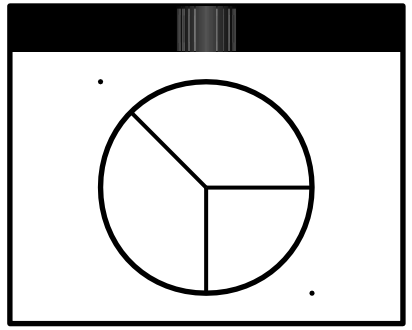
CMPUT 360 – Introduction to Software Engineering

Slides adapted from Henry Tang, Dr. Abram Hindle

Views







Need to maintain consistency in the views

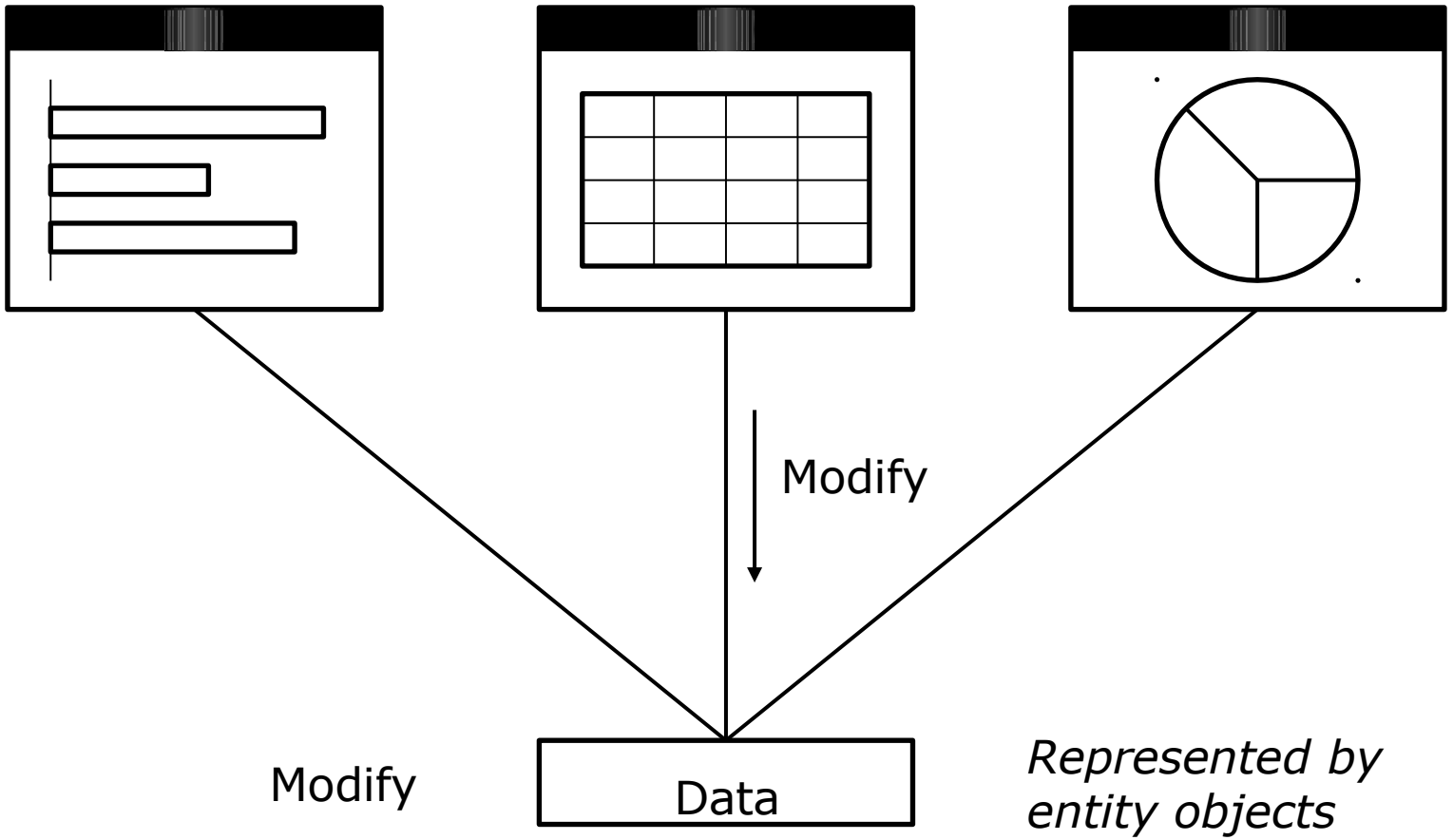
Want clear, separate responsibilities for presentation, interaction, computation, and representation

Data

Need to update multiple views of the common data model

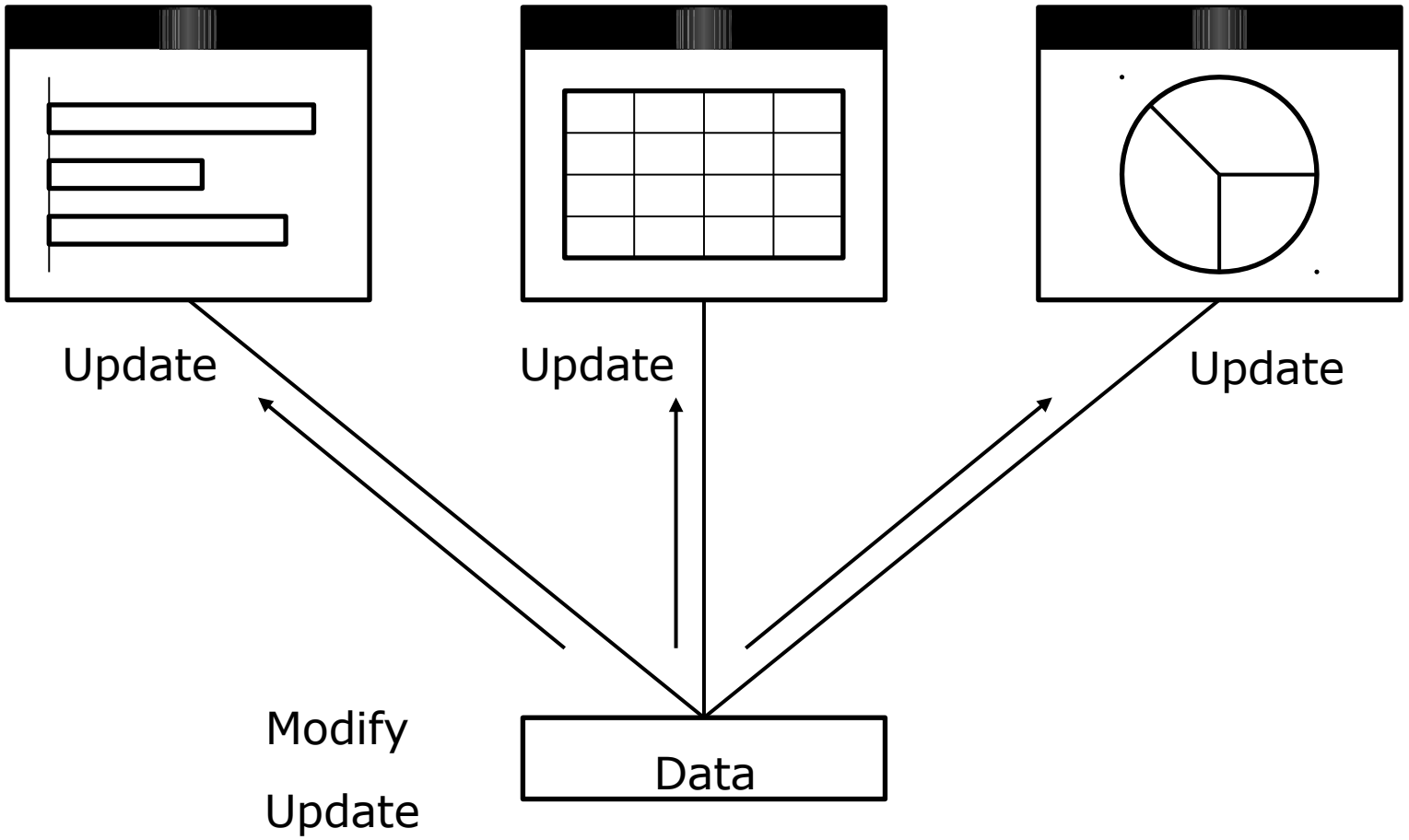
Model

Views (i.e., observers, clients)



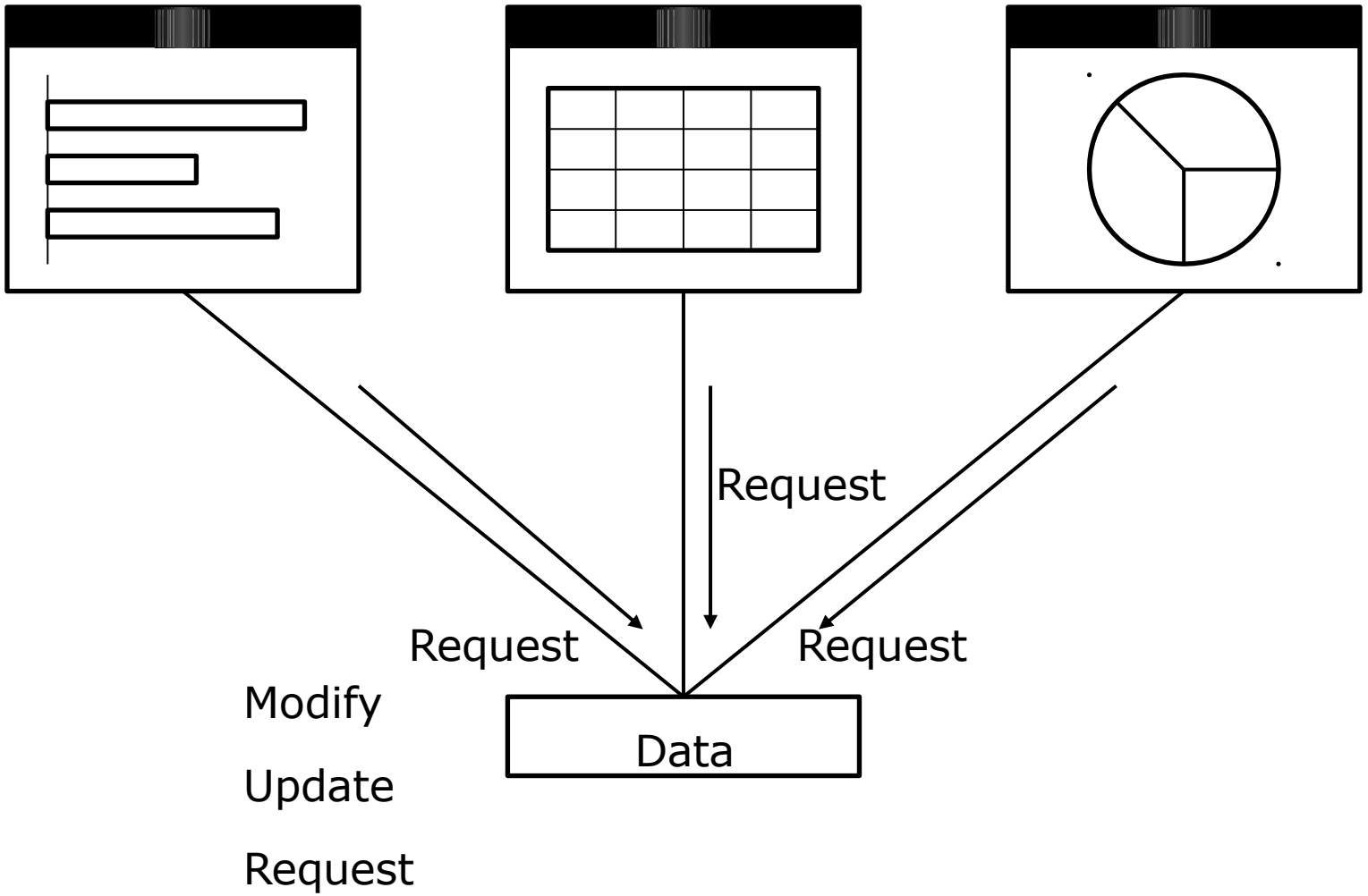
Model (i.e., subject, server)

Views (i.e., observers, clients)



Model (i.e., subject, server)

Views (i.e., observers, clients)



Model (i.e., subject, server)

Model/View/Controller Roles

- Model:
  - Entity layer
    - Complete, self-contained representation of the data managed by the application
    - Provides services to manipulate this data
    - “The back end”
  - Main responsibilities
    - Representation and computation issues
    - Sometimes persistence

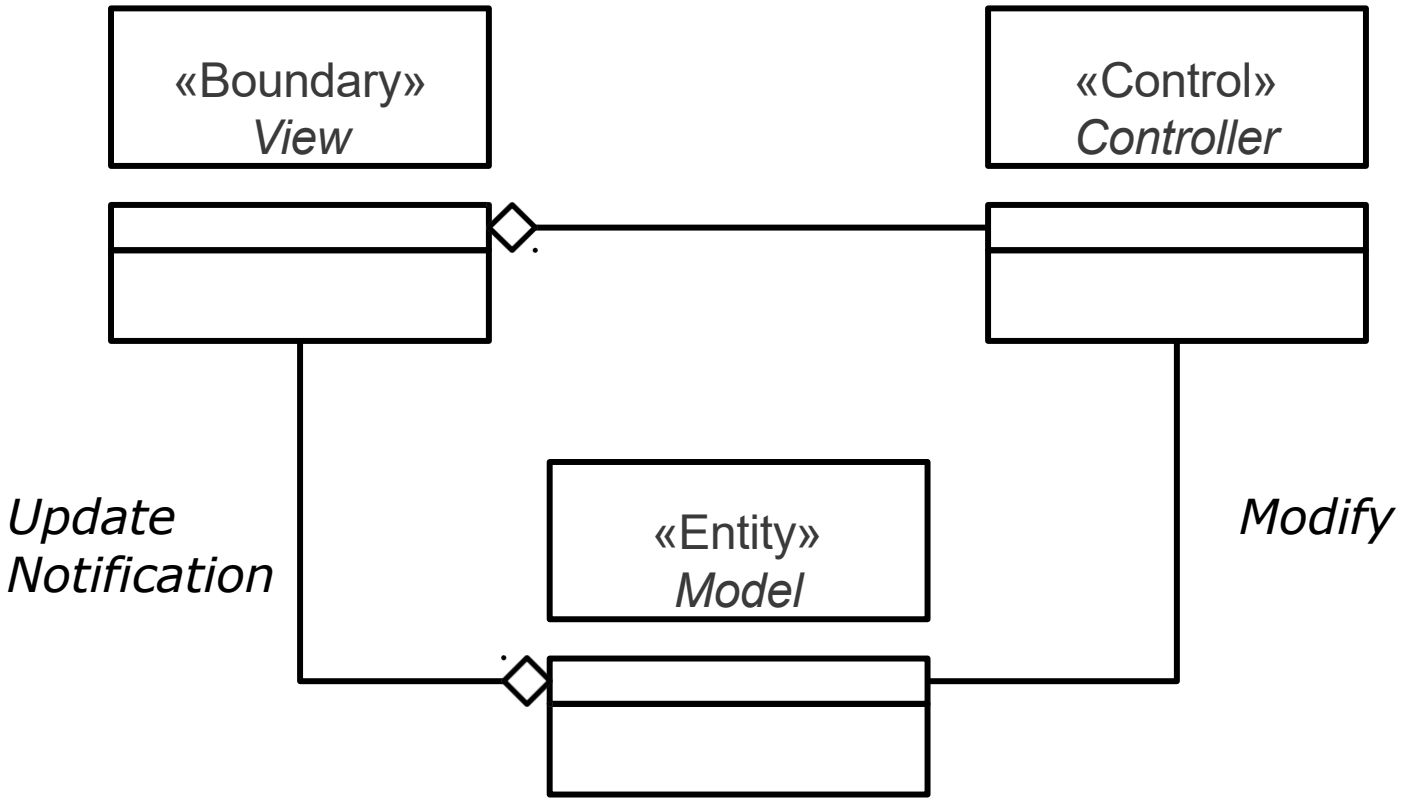
MVC Roles

- View:
  - Boundary layer
    - Set of user interface components determines what is needed for a particular perspective of the data
    - “The front end”
  - Main responsibility
    - Presentation issues

MVC Roles

- Controller:
  - Control layer
    - Handles events and uses appropriate information from user interface components to modify the model
  - Main responsibility
    - Interaction issues

Can create new, specific types of views without changing the model



The model should not need to know the particulars of a specific view

The model should not need to know about any controllers

MVC Design Issues

- Swing dependent part:
  - Views contain Swing components
  - Controllers are Swing listeners
- Swing independent part:
  - The model should be as Swing-free as possible
    - E.g., not using Swing types in entity classes

“MV” Design

- Generalization:
  - Use “model” superclass and “view” interface
    - All models keep track of their views
    - When changed, all models notify their views to update
    - All views update themselves when notified
  - Have application-specific model and view classes

Java Observer – java.util.Observable superclass

```
public class Observable {
        ArrayList Observale
        public Observable() { … }

        // “all models keep track of their views”
        public void addObserver( Observer o ) { … }
        public void deleteObserver( Observer o ) { … }

        // “all models notify their views to update”
        public void notifyObservers() { … }
        public void notifyObservers( Object arg ) { … }

        // note whether the model has changed
        public boolean hasChanged() { … }
        protected void clearChanged() { … }
        protected void setChanged() { … }
        …
}
```

Java Observer – java.util.Observer interface

```
public interface Observer {
     public void update( Observable s, Object arg );
}
```

Java Observer

```
// MyModel.java
import java.util.*;

public class MyModel extends Observable {
	private String message;

	public MyModel() {
		message = “”;
	}
	public String getMessage() {
		return message;
	}
	public void setMessage( String message ) {
		this.message = message;
		setChanged();
		notifyObservers(); // clears changed flag
	}
}
```

Java Observer

```
// MyView.java
import java.util.*;

public class MyView implements Observer {
     public void update( Observable s, Object arg ) {
          System.out.println(
               ((MyModel) s).getMessage()
          );
     }
}
```

Java Observer

```
// MyApp.java

public class MyApp {

	public static void main( String args[] ) {

		MyModel theModel = new MyModel();
		MyView aView = new MyView();
		MyView anotherView = new MyView();

		theModel.addObserver( aView );
		theModel.addObserver( anotherView );

		theModel.setMessage( "hello" );
	}
}
```

MVC Design

- Approach:
  - Use a framework that supports MVC to help structure an interactive application
  - Framework is a set of cooperating classes that forms a reusable design in a particular domain
  - Reusable design *and* code

MVC Framework

Who is in Control?

- Class library reuse
  - Application developers:
    - Write the main body of the application
    - Reuse library code by calling it
- Framework reuse
  - Application developers:
    - Reuse the main body of the application
    - Write code that the framework calls
    - Reuse library code by calling it

Framework

- Separation of concerns:
  - Framework
    - Skeletal application code
    - General superclasses and interfaces
  - Your “customizations”
    - Specific subclasses and implementations